VPN-LAN svord Bridge — Operator Setup Guide

Revision: 1 **Last modified:** 2026-07-01T16:00:22Z **Status:** Active — operator manual for the VPN-LAN service-access feature (Phase 9 of `../design/vpn_lan_access/PLAN.md`) **Authority:** Inherits `constitution/Constitution.md` per §11.4.35. Consumer-side operator guide for the env-var svord bridge contract; the design source-of-truth is `PLAN.md` §3 (contract) + §2 (routing map) + §4 (security) and the `miracast_verdict.md` structural verdict. **Feature workstream:** `feature/vpn-aware-dynamic-routing` (§11.4.167)

---

1. Overview — what this feature does

This feature lets you **reach and use mainstream services that live on a VPN-internal LAN through helix_proxy**. helix_proxy connects to that remote network by driving a sibling **bridge project** (in our deployment: `svord_toolkit`) over its own `svord-ssh-*` scripts, and then routes / proxies / reflects each service so it becomes usable from the helix_proxy side.

helix_proxy never hardcodes the bridge project. It reads a small **bridge contract** of 6 environment variables (§4 below), invokes only the operator-supplied hooks, and never reaches into the bridge project's internals or modifies it or any remote host (invocation-only, §11.4.122).

The one-line routing map

The VPN is **L3-routed** (WireGuard + L2TP/PPP over `ppp0`; reachable subnet `10.0.0.0/8`). That single fact decides how every protocol is carried (`PLAN.md` §2):

- **Unicast services ROUTE** over the L3 VPN (SMB, NFS, FTP-control, SFTP, IMAP, SMTP, POP3, Cast control, DIAL HTTP).
- **HTTP-shaped services are PROXIED** through the **existing Squid** (WebDAV and any other HTTP origin) — no new component required.
- **Multicast discovery is REFLECTED** on the remote (device-side) network, because routers do not forward multicast across L3 (mDNS, SSDP/UPnP, WS-Discovery, DNS-SD, Chromecast/DIAL discovery).

- **L2 / Wi-Fi-Direct is STRUCTURALLY-IMPOSSIBLE** over an L3 VPN — this is **Miracast** (§7 + `miracast_verdict.md`, §11.4.112).

Rule of thumb: unicast services route; HTTP-shaped services already work through Squid; multicast discovery needs a remote-side reflector; Wi-Fi-Direct / L2 is out of scope by physics.

---

2. Prerequisites

Before you configure the bridge you need:

- **The sibling bridge project** — in our deployment `svord_toolkit`, checked out beside helix_proxy (default example location `../svord_toolkit`). It must provide the three `svord-ssh-*` scripts the contract points at:
  - `svord-ssh-connect` — brings the VPN link **up**,
  - `svord-ssh-disconnect` — tears the VPN link **down**,
  - `svord-ssh-health` — reports reachability (**exit 0 == up**, non-zero == down).
- **A running base proxy** — the helix_proxy Squid/Dante stack must be up, because HTTP-shaped traffic (e.g. WebDAV) is carried through the **existing Squid** and the SSRF allowlist carve-out is applied to the existing Dante/Squid config (§6).
- **A `ping` or `nc` tool on the host** — `scripts/svord_doctor.sh` uses one of them for the remote-host smoke probe. If neither is present the doctor cannot confirm reachability and reports `SKIP:host_probe_unavailable` (§5).

> The live VPN connection itself (secrets, credentials, root/sudo access on the bridge side) is **operator-supplied and operator-gated**. Everything in this guide that does *not* require the live connection — the doctor, the honest-SKIP path, the config logic — is exercisable without any secrets.

---

3. The bridge contract at a glance

helix_proxy reads **6** variables. Real values live in a **gitignored `.env`**; a tracked **`.env.example`** documents the shape (the §11.4.77 re-obtain mechanism). The example values below are **illustrative only** — they carry names and paths, never secrets.

Env var	Meaning	Illustrative example (from `.env.example`)
`HELIX_SVORD_DIR`		`../svord_toolkit`

Env var	Meaning	Illustrative example (from `.env.example`)
	Path to the sibling bridge project	
`HELIX_BRIDGE_CONNECT`	Command that brings the VPN **up** (first token must be an executable file)	`${HELIX_SVORD_DIR}/svord-ssh-connect`
`HELIX_BRIDGE_DISCONNECT`	Command that tears the VPN **down** (first token executable)	`${HELIX_SVORD_DIR}/svord-ssh-disconnect`
`HELIX_BRIDGE_HEALTH`	Health probe — **exit 0 == up**, the authoritative up/down signal	`${HELIX_SVORD_DIR}/svord-ssh-health`
`HELIX_BRIDGE_SUBNET`	Reachable remote subnet (CIDR) — for route scoping + the narrow SSRF allowlist carve-out	`10.0.0.0/8`
`HELIX_BRIDGE_HOST`	A known remote host inside the subnet, for smoke reachability	`10.6.100.221`

`HELIX_SVORD_DIR` is the **only** place the bridge-project name lives — change it there to point at a different bridge and nothing else needs editing (§11.4.28 decoupled).

---

4. Setup steps

Step 1 — copy the template to a gitignored `.env`

From the helix_proxy repo root:

```
cp .env.example .env
```

`.env` is gitignored (line 6 of `.gitignore`) and **must never be committed** (§11.4.10 / §11.4.30). The template carries **names and illustrative paths only** — put no secrets in `.env.example`, ever.

Step 2 — fill in the 6 bridge vars

Edit `.env` and point the 6 `HELIX_SVORD_DIR` / `HELIX_BRIDGE_*` vars at your real bridge project (see the table in §3). Rules that the doctor enforces:

- Each of `HELIX_BRIDGE_CONNECT` / `HELIX_BRIDGE_DISCONNECT` / `HELIX_BRIDGE_HEALTH` must have a **first whitespace token that is an existing, executable file** — otherwise the contract is `MISCONFIGURED` (§5).
- `HELIX_BRIDGE_SUBNET` is the CIDR later used for L3 route scoping **and** the narrow SSRF allowlist carve-out (§6). Keep it as tight as your deployment allows.
- `HELIX_BRIDGE_HOST` must be a real host inside that subnet — the doctor pings / `ncs` it as the final smoke check.

All 6 vars must be **set and non-empty**; any unset/empty var makes the contract unresolvable (`MISCONFIGURED:env_unset`).

Step 3 — run the doctor and read the `BRIDGE:` verdict

`scripts/svord_doctor.sh` is the preflight. Every downstream VPN-LAN test runs it first. It reads the contract **from the environment**, so source your `.env` first:

```
set -a; . ./.env; set +a
scripts/svord_doctor.sh
```

The doctor prints diagnostic `svord-doctor:` lines and then exactly **one** final verdict line matching `^BRIDGE: `. It **never fakes UP**: a genuine `UP` requires the authoritative health probe to exit 0 **and** the remote host to be reachable.

Final verdict line	Meaning	Exit code
`BRIDGE: UP`	Contract resolved, hooks executable, `HELIX_BRIDGE_HEALTH` exit 0, remote host reachable — the VPN is genuinely up	`0`
`BRIDGE: SKIP:network_unreachable_external`	Contract fine, but the health probe reported **down** or the host is unreachable — the bridge is down	`2`

Final verdict line	Meaning	Exit code
`BRIDGE: SKIP:host_probe_unavailable`	Health probe up, but no `ping`/`nc` tool exists to confirm host reachability — cannot confirm	2
`BRIDGE: MISCONFIGURED:<reason>`	Bad contract — e.g. `env_unset`, `hook_not_executable:<hooks>`, `bridge_lib_missing` — fix the `.env` and re-run	3

Exit codes at a glance: **`0` = up, `2` = down / operator-blocked (honest SKIP), `3` = misconfigured**. Wire your automation to treat `2` as SKIP (not failure) and `3` as a setup error to fix.

---

5. Honest-SKIP behaviour — by design, not a failure

When the bridge is **down**, every VPN-LAN test **honestly SKIPs** with the closed-set reason `network_unreachable_external` (§11.4.3) and **never** reports a fake PASS. This is **deliberate**: the live VPN connection is operator-supplied, so the autonomous slate must not manufacture a green result for a path it cannot actually exercise.

The mechanism is the same in both the script and the library:

- `scripts/svord_doctor.sh` emits `BRIDGE: SKIP:network_unreachable_external` and exits `2` when the health probe reports down or the host is unreachable.
- `tests/lib/svord_bridge.sh` exposes `bridge_require`, the gate every downstream test calls: it returns `0` when up; when down it echoes `SKIP:network_unreachable_external` and returns `2` (OPERATOR-BLOCKED per §11.4.68 / §11.4.69); when the contract is unset it echoes `SKIP:misconfigured` and returns `3`.

So a SKIP means "the bridge is down (or unconfigured), and the harness told you the truth about it" — exit `2` is an honest OPERATOR-BLOCKED signal, **not** a red test. A metadata-only, config-only, or absence-of-error PASS on a path the bridge cannot reach would be a §11.4 bluff; the honest SKIP is what keeps that from happening.

---

6. Security notes

Widening egress to the VPN subnet reopens SSRF surface, so the bridge is deployed with the existing hardening kept intact (PLAN.md §4):

- **RFC1918 / link-local / loopback / metadata block stays as the floor.** The prior Dante SSRF + Squid ACL hardening is **not removed**. Every private range other than the bridge subnet stays denied.
- **Only a narrow allowlist carve-out** is added for `HELIX_BRIDGE_SUBNET`. In the Dante first-match top-down order the factory-subnet ALLOW rule sits **above** the broad internal-DENY, so exactly that one subnet is reachable and every other RFC1918 range remains blocked. Keep `HELIX_BRIDGE_SUBNET` as tight as your deployment allows — it is the width of the hole.
- **Email open-relay guard.** helix_proxy must **never expose an anonymous CONNECT to :25.** Authenticated **submission** (587/465) is routed to VPN clients; server-to-server `:25` stays behind the boundary. The SSRF allowlist must not turn helix_proxy into an anonymizing spam conduit. Prefer implicit-TLS ports (993/995/465, RFC 8314) over plaintext-upgradable STARTTLS ports where you have the choice.
- **No secrets in git.** `.env` is gitignored and never committed; `.env.example` carries names and paths only; credentials for the live connection live outside the tree (§11.4.10 / §11.4.30). Run the pre-store leak audit (§11.4.10.A) before storing any operator-supplied secret.

Each egress widening ships with a paired mutation proving the allowlist has teeth (an out-of-allowlist target still denies), so the carve-out cannot silently become permissive.

---

7. Protocol support matrix

How each in-scope protocol is carried, and its status. **Approach** follows the §1 routing map; unicast services **route** over the L3 VPN, HTTP-shaped services are **proxied via the existing Squid**, multicast discovery is **reflected at the remote** site, and Wi-Fi-Direct is **structurally-impossible**.

Protocol / service	Approach	Status
SMB / CIFS / NMB (NetBIOS)	Route unicast over L3 (needs L3 routing, **not** SOCKS5); NMB name resolution via unicast fallback	Designed — round-trip proven when bridge up; honest SKIP when down

Protocol / service	Approach	Status
NFS	Route unicast over L3 (2049 + aux)	Designed — read/write round-trip when bridge up; honest SKIP when down
FTP / FTPS	Route control (21) + the server's pinned passive-port range; FTPS explicit (`AUTH TLS`) + implicit (990)	Designed — passive round-trip when bridge up; honest SKIP when down
SFTP	Route over SSH (22) — the recommended modern single-connection path	Designed — byte round-trip when bridge up; honest SKIP when down
WebDAV	**Proxy via the existing Squid** (HTTP `PROPFIND`/`MKCOL`) — no new component	Designed — 207/201 bodies when bridge up; honest SKIP when down
IMAP / IMAPS	Route unicast (993)	Designed — mailbox LIST when bridge up; honest SKIP when down
SMTP / submission	Route authenticated submission (587/465) — **never** anonymous CONNECT-:25 (§6)	Designed — `250` accepted + open-relay negative test when bridge up; honest SKIP when down
POP3 / POP3S	Route unicast (995)	Designed — retrieve when bridge up; honest SKIP when down
mDNS / SSDP / WS-Discovery / DNS-SD / DIAL discovery	**Reflect at the remote** site (Avahi reflector / SSDP relay) — routers do not forward multicast across L3; **operator-gated** remote deployment (§11.4.122)	Designed — reflector operator-gated; honest SKIP when not deployed
Chromecast (Google Cast / DIAL)	Discovery via the remote reflector; **control routes** as unicast TCP (8008 / 8009 TLS, DIAL HTTP)	Designed — eureka_info JSON + cast status transition when bridge up; honest SKIP when no device/reflector
	Route unicast TCP (5555), central adb-	Designed — remote serial + `getprop` when

Protocol / service	Approach	Status
ADB (access / debug / connect)	server model — no proxy hop	bridge up; honest SKIP when down
ADB flash (`fastboot`)	USB-level → route via **`usbip`** (USB-over-IP); network `fastboot` is honestly USB-bound; operator-gated on real hardware (§11.4.133)	Honest boundary — usbip smoke or honest SKIP; operator-gated
Miracast	**STRUCTURALLY-IMPOSSIBLE** over an L3 VPN — Wi-Fi-Direct / L2, no routable IP hop (§11.4.112)	**Won't-fix: structurally-impossible** — see `miracast_verdict.md`; use Chromecast instead

The **Miracast** row is authoritative per `miracast_verdict.md`: Miracast's transport is a Wi-Fi Direct Layer-2 radio P2P group with no routable IP hop, so an L3 VPN has nothing to route. The routable "cast to a remote display" capability is delivered instead by **Google Cast / DIAL** (the Chromecast row above).

---

8. FAQ

Q1. The doctor says `SKIP:network_unreachable_external` — is that a test failure? No. It means the **bridge is down** (the health probe reported non-zero, or the remote host was unreachable). The doctor exits `2`, which is an honest OPERATOR-BLOCKED SKIP, not a red result. Bring the VPN up (or fix connectivity) and re-run; the live round-trip evidence only exists when the bridge is genuinely up (§5).

Q2. Can I cast to a remote display via Miracast? No — Miracast **cannot traverse the L3 VPN**. It rides Wi-Fi Direct at Layer 2 (a direct radio P2P group between two devices in RF proximity) with no routable IP hop for the VPN to carry — a structural impossibility of the standard, not a missing feature (`miracast_verdict.md`, §11.4.112). Use **Google Cast / DIAL (Chromecast)** instead: its control plane routes over the VPN as ordinary unicast TCP, and only its discovery needs the remote-side reflector.

Q3. Does WebDAV need a new service? No. WebDAV is HTTP-shaped, so it goes through the **existing Squid** (`PROPFIND` / `MKCOL`); no new component is added. On an older Squid you may need to enable `extension_methods`, and the WebDAV origin's TLS port must be in Squid's `SSL_Ports` for `CONNECT`.

Q4. Why do mounts (SMB / NFS) need L3 routing instead of SOCKS5? Because they are **unicast IP services that route** over the L3 VPN — SOCKS5/Squid is the wrong primitive for mounts and discovery. The VPN is L3-routed, so the correct primitive is a routed gateway plus a scoped SSRF allowlist, not a SOCKS5 hop (`PLAN.md` §2).

Q5. Why does discovery (mDNS / SSDP) need a reflector on the remote side? Multicast discovery is **not forwarded across L3** by routers. To enumerate a remote-site service you deploy a remote-side reflector (Avahi `enable-reflector=yes` for mDNS, an SSDP relay for 1900). That deployment changes a remote host, so it is **operator-gated** — it only happens after an interactive question with options (§11.4.122).

Q6. Can I flash a device over the VPN with `fastboot`? Not directly — `fastboot` is USB-level, not a routable IP service. It is carried via **`usbip`** (USB-over-IP) from a remote host that has the device physically attached; network `fastboot` is honestly USB-bound. Any real-device flash is **operator-gated** for target-hardware safety (§11.4.133 / §11.4.122).

Q7. Will widening egress to the VPN subnet weaken the SSRF hardening? No — the RFC1918 / link-local / loopback / metadata **block stays as the floor**, and only a **narrow carve-out** for `HELIX_BRIDGE_SUBNET` is added above the internal-deny in Dante's first-match order. Every other private range stays denied, and a paired mutation proves the allowlist has teeth (§6).

Q8. Where do the live secrets go — do they belong in `.env.example`? Never. `.env.example` carries **names and illustrative paths only**. Real values go in the gitignored `.env` (which is never committed), and the live-connection credentials live **outside the tree** entirely (§11.4.10 / §11.4.30). Run the pre-store leak audit before storing any operator-supplied secret (§11.4.10.A).

Sources

- `../design/vpn_lan_access/PLAN.md` — the comprehensive phased plan: §2 routing map, §3 env-var bridge contract, §4 security reconciliation, §5 per-phase protocol coverage.
- `../design/vpn_lan_access/miracast_verdict.md` — the §11.4.112 structural-impossibility verdict for Miracast (authoritative for

the Miracast row in §7 and FAQ Q2), with cited Wi-Fi Alliance / Wi-Fi Direct evidence.
- `.env.example` (VPN-LAN SVORD BRIDGE CONTRACT section) — the tracked, no-secrets template for the 6-var contract copied into `.env`.
- `scripts/svord_doctor.sh` — the preflight doctor: the `BRIDGE:` verdict lines and exit codes `0` / `2` / `3` documented in §4.
- `tests/lib/svord_bridge.sh` — the sourceable contract library: `bridge_load`, `bridge_up`, `bridge_require`, `bridge_subnet`, `bridge_host`.